Data Streaming: Crypto Market Data

Evan G.

Cloud Data Architect | *github: evan-gloria* | *linkedin: efgloria*

Published March 2026

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

© 2026 Evan G. | Data Streaming: Crypto Market Data | Portfolio Project

Data Streaming: Crypto Market Data

The Problem: High-velocity market streams cause API connection exhaustion, while traditional S3 data lakes suffer from "invisible data" requiring constant manual partition repairs.

The Solution: A fully serverless ingestion engine and Medallion Data Lake powered by Kinesis Firehose, AWS Glue, and Apache Iceberg for true ACID transactions.

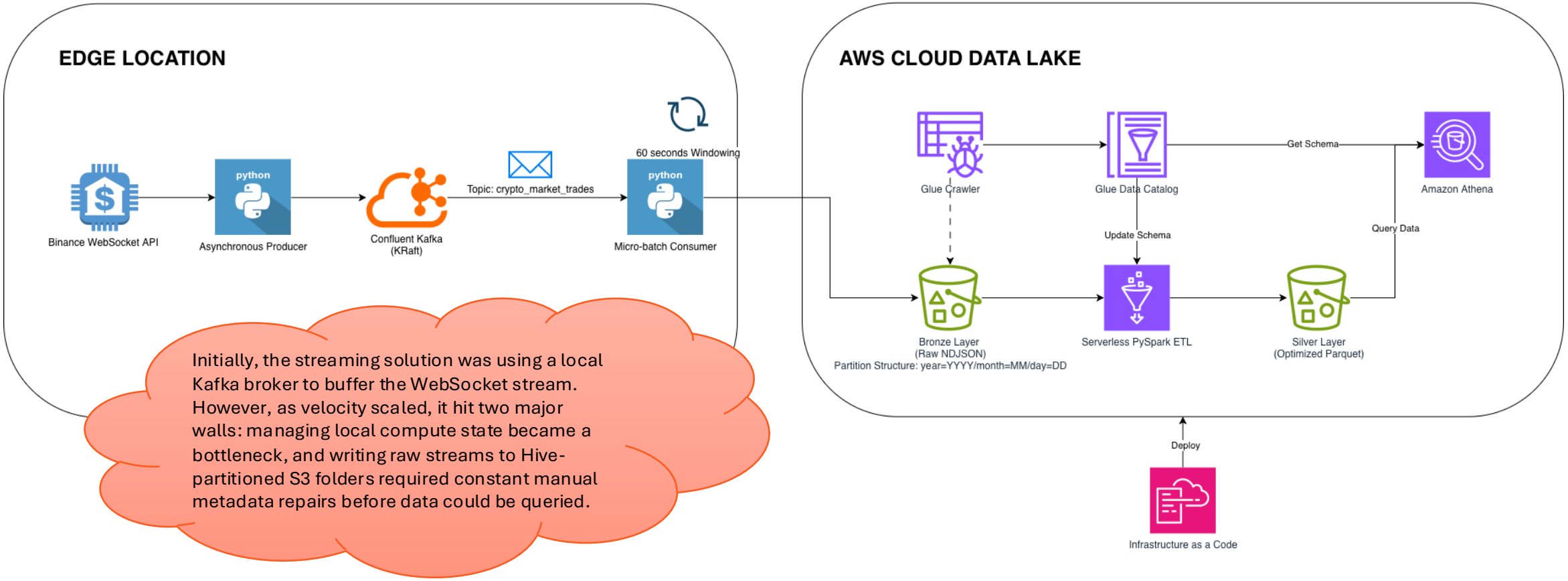
The Benefit: Eliminates infrastructure overhead and unlocks instant SQL access to production-ready data with zero manual maintenance.

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

© 2026 Evan G. | Data Streaming: Crypto Market Data | Portfolio Project

Local Kafka and Parquet Architecture (V1)

CRYPTO MARKET TRADE POC



Initially, the streaming solution was using a local Kafka broker to buffer the WebSocket stream. However, as velocity scaled, it hit two major walls: managing local compute state became a bottleneck, and writing raw streams to Hive-partitioned S3 folders required constant manual metadata repairs before data could be queried.

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

The Architectural Pivot

Legacy State (Local Kafka & Parquet):

Compute: Relied on local hardware to run the message broker.

Storage: Required manual MSCK REPAIR TABLE commands to make newly streamed data visible to Amazon Athena.

Current State (Serverless & Iceberg):

Compute: Containerized Python producer deployed to AWS Fargate (0.25 vCPU) for 24/7 headless execution.

Ingestion: Kinesis Firehose buffers and micro-batches records to solve latency bottlenecks and eliminate broken pipes.

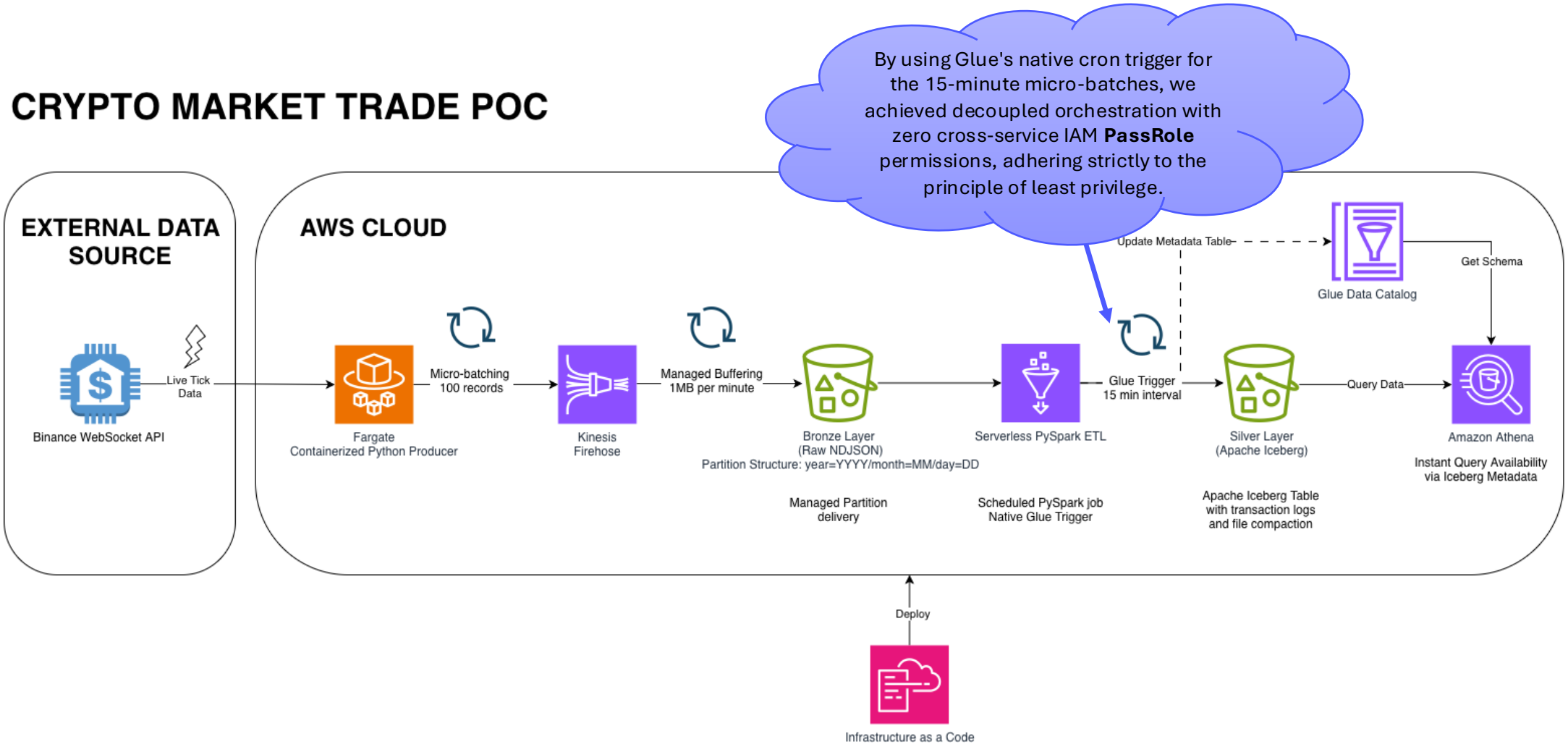
Storage: Apache Iceberg replaces folder-based tracking with file-level transaction logs, completely automating metadata management.

Orchestration: A native AWS Glue Scheduled Trigger automates the 15-minute Silver ETL micro-batches with zero cross-service IAM overhead.

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

© 2026 Evan G. | Data Streaming: Crypto Market Data | Portfolio Project

Serverless and Iceberg Architecture (V2)



DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

© 2026 Evan G. | Data Streaming: Crypto Market Data | Portfolio Project

Outcome & Results

Cost Optimization: Fargate footprint minimized (0.25 vCPU) and ETL batched to 15-minute intervals, slashing compute overhead compared to real-time event-driven processing.

Zero Maintenance Operations: The combination of managed Firehose buffering and Apache Iceberg completely eliminated the need for infrastructure management and partition repairs.

Time-to-Insight: Data analysts can unlock instant SQL access to production-ready data with guaranteed schema enforcement immediately after a micro-batch completes.

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.

Links:

General API Reference:

- Binance Official Spot API Documentation: https://developers.binance.com/docs/binance-spot-api-docs

Specific WebSocket Reference (Best for this POC):

- Binance Official Market Data WebSocket Streams: https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams

GitHub Repo:

- GitHub: https://github.com/evan-gloria/crypto-market-data-pipeline

DISCLAIMER: This solution architecture was built as a proof-of-concept for personal portfolio and educational purposes. It is not affiliated with, endorsed by, or connected to Binance or any financial institution. All data ingested is from publicly available WebSocket streams, and this project does not constitute financial or trading advice.